CHAPTER 18

THE PHYSICAL DESIGN PROCESS

CHAPTER OBJECTIVES

- Distinguish between physical design and logical design as applicable to the data warehouse
- Study the steps in the physical design process in detail
- Understand physical design considerations and know the implications
- Grasp the role of storage considerations in physical design
- Examine indexing techniques for the data warehouse environment
- Review and summarize all performance enhancement options

As an IT professional, you are familiar with logical and physical models. You have probably worked with the transformation of a logical model into a physical model. You also know that completing the physical model has to be tied to the details of the computing platform, the database software, hardware, and any third-party tools.

As you know, in an OLTP system you have to perform a number of tasks for completing the physical model. The logical model forms the primary basis from which the physical model is derived. But, in addition, a number of factors must be considered before you can get to the physical model. You must determine where to place the database objects in physical storage. What is the storage medium and what are its features? This information helps you define the storage parameters. Then you have to plan for indexing, an important consideration. On which columns in each table must the indexes be built? You need to look into other methods for improving performance. You have to examine the initialization parameters in the DBMS and decide how to set them. Similarly, in the data warehouse environment, you need to consider many different factors to complete the physical model.

Data Warehousing Fundamentals for IT Professionals, Second Edition. By Paulraj Ponniah
Copyright © 2010 John Wiley & Sons, Inc.

We have considered the logical model for the data warehouse in sufficient detail. You have mastered the dimensional modeling technique that helps you design the logical model. In this chapter, we will use the logical model of a data warehouse to develop and complete the physical model. Physical design gets the work of the project team closer to implementation and deployment. Every task so far has brought the project to the grand logical model. Now, physical design moves it to the next significant phases.

PHYSICAL DESIGN STEPS

Figure 18-1 is a pictorial representation of the steps in the physical design process for a data warehouse. Note the steps indicated in the figure. In the following subsections, we will broadly describe the activities within these steps. You will understand how at the end of the process you arrive at the completed physical model. After the end of this section, the rest of the chapter elaborates on all the crucial aspects of the physical design.

Develop Standards

Many companies invest a lot of time and money to prescribe standards for information systems. The standards range from how to name the fields in the database to how to conduct interviews with the user departments for requirements definition. A group in IT is designated to keep the standards up to date. In some companies, every revision must be updated and authorized by the CIO. Through the standards group, the CIO ensures that the standards are followed correctly and strictly. Usually the practice is to publish the standards on the company's intranet. If your IT department is one of the progressive ones giving due attention to standards, then be happy to embrace and adapt the standards for the data warehouse.

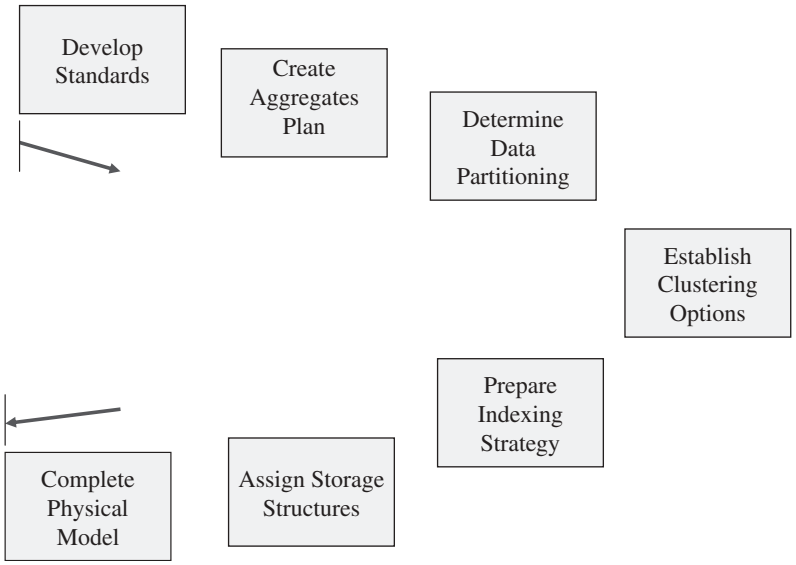


Figure 18-1 The physical design process.

In the data warehouse environment, the scope of the standards expands to include additional areas. Standards ensure consistency across the various areas. If you have the same way of indicating names of the database objects, then you are leaving less room for ambiguity. Let us say the standards in your company require the name of an object to be a concatenation of multiple words separated by dashes and that the first word in the group indicates the business subject. With these standards, as soon as someone reads an object name, that person can know the business subject.

Standards take on greater importance in the data warehouse environment. This is because the use of the object names is not confined to the IT department. Users will also be referring to the objects by names when they formulate and run their own queries. As standards are quite significant, we will come back to them a little later in this chapter. Now let us move on to the next step in the physical design.

Create Aggregates Plan

Let us say that in your environment more than 80% of the queries ask for summary information. If your data warehouse stores data only at the lowest level of granularity, every such query has to read through all the detailed records and sum them up. Consider a query looking for total sales for the year, by product, for all the stores. If you have detailed records keeping sales by individual calendar dates, by product, and by store, then this query needs to read a large number of detailed records. So, what is the best method to improve performance in cases like this? If you have higher levels of summary tables of products by store, the query could run faster. But how many such summary tables must you create? What is the limit?

In this step, review the possibilities for building aggregate or summary tables. You get clues from the requirements definition. Look at each dimension table and examine the hierarchical levels. Which of these levels are more important for aggregation? Clearly assess the trade-off. What you need is a comprehensive plan for aggregation. The plan must spell out the exact types of aggregates you must build for each level of summarization. It is possible that many of the aggregates will be present in the OLAP system. If OLAP instances are not for universal use by all users, then the necessary aggregates must be present in the main warehouse. The aggregate database tables must be laid out and included in the physical model. We will have some more to say about summary levels in a later section.

Determine the Data Partitioning Scheme

Consider the data volumes in the warehouse. What about the number of rows in a fact table? Let us make some rough calculations. Assume there are four dimension tables with 50 rows each on average. Even with this limited number of dimension table rows, the potential number of fact table rows exceeds six million. Fact tables are generally very large. Large tables are not easy to manage. During the load process, the entire table must be closed to users. Again, back up and recovery of large tables pose difficulties because of their sheer sizes. Partitioning divides large database tables into manageable parts.

Always consider partitioning options for fact tables. It is not just the decision to partition that counts. Based on your environment, the real decision is about how exactly to partition the fact tables. Your data warehouse may be a conglomerate of conformed data marts. You must consider partitioning options for each fact table. Should some be partitioned vertically

and the others horizontally? You may find that some of your dimension tables are also candidates for partitioning. Product dimension tables are especially large. Examine each of your dimension tables and determine which of these must be partitioned.

In this step, come up with a definite partitioning scheme. The scheme must include:

- The fact tables and the dimension tables selected for partitioning
- The type of partitioning for each table—horizontal or vertical
- The number of partitions for each table
- The criteria for dividing each table (for example, by product groups)
- A description of how to make queries aware of partitions

Establish Clustering Options

In the data warehouse, many of the data access patterns rely on sequential access of large quantities of data. Whenever you have this type of access and processing, you will realize much performance improvement from clustering. This technique involves placing and managing related units of data in the same physical block of storage. This arrangement causes the related units of data to be retrieved together in a single input operation.

You need to establish the proper clustering options before completing the physical model. Examine the tables, table by table, and find pairs that are related. This means that rows from the related tables are usually accessed together for processing in many cases. Then make plans to store the related tables close together in the same file on the storage medium. For two related tables, you may want to store the records from both files interleaved. A record from one table is followed by all the related records in the other table while storing in the same physical file.

Prepare an Indexing Strategy

This is a crucial step in the physical design. Unlike OLTP systems, the data warehouse is query-centric. As you know, indexing is perhaps the most effective mechanism for improving performance. A solid indexing strategy results in enormous benefits. The strategy must lay down the index plan for each table, indicating the columns selected for indexing. The sequence of the attributes in each index also plays a critical role in performance. Scrutinize the attributes in each table to determine which attributes qualify for bitmapped indexes.

Prepare a comprehensive indexing plan. The plan must indicate the indexes for each table. Further, for each table, present the sequence in which the indexes will be created. Describe the indexes that are expected to be built in the very first instance of the database. Many indexes can wait until you have monitored the data warehouse for some time. Spend enough time on the indexing plan.

Assign Storage Structures

Where do you want to place the data on the physical storage medium? What are the physical files? What is the plan for assigning each table to specific files? How do you want to divide each physical file into blocks of data? Answers to questions like these go into the data storage plan.

In an OLTP system, all data resides in the operational database. When you assign the storage structures in an OLTP system, your effort is confined to the operational tables accessed by the user applications. In a data warehouse, you are not just concerned with the physical files for the data warehouse tables. Your storage assignment plan must include other types of storage such as the temporary data extract files, the staging area, and any storage needed for front-end applications. Let the plan include all the types of storage structures in the various storage areas.

Complete Physical Model

This final step reviews and confirms the completion of the prior activities and tasks. By the time you reach this step, you have the standards for naming the database objects. You have determined which aggregate tables are necessary and how you are going to partition the large tables. You have completed the indexing strategy and have planned for other performance options. You also know where to put the physical files.

All the information from the prior steps enables you to complete the physical model. The result is the creation of the physical schema. You can code the data definition language statements (DDL) in the chosen RDBMS and create the physical structure in the data dictionary.

PHYSICAL DESIGN CONSIDERATIONS

We have traced the steps for the physical design of the data warehouse. Each step consists of specific activities that finally lead to the physical model. When you look back at the steps, one step relates to the physical storage structure and several others deal with the performance of the data warehouse. Physical storage and performance are significant factors. We will cover these two in sufficient depth later in the chapter.

In this section, we will firm up our understanding of the physical model itself. Let us review the components and track down what it takes to move from the logical model to the physical model. First, let us begin with the overall objectives of the physical design process.

Physical Design Objectives

When you perform the logical design of the database, your goal is to produce a conceptual model that reflects the information content of the real-world situation. The logical model represents the overall data components and the relationships. The objectives of the physical design process do not center on the structure. In physical design, you are getting closer to the operating systems, the database software, the hardware, and the computing platform. You are now more concerned about how the model is going to work than on how the model is going to look.

If you want to summarize, the major objectives of the physical design process are improving performance on the one hand, and improving the management of the stored data on the other. You base your physical design decisions on the usage of data. The frequency of access, the data volumes, the specific features supported by the chosen RDBMS, and the configuration of the storage medium influence the physical design decisions. You need to

pay special attention to these factors and analyze each to produce an efficient physical model. Now let us present the significant objectives of physical design.

Improve Performance Performance in an OLTP environment differs from that of a data warehouse in the online response times. Whereas a response time of less than three seconds is almost mandatory in an OLTP system, the expectation in a data warehouse is less stringent. Depending on the volume of data processed during a query, response times varying from a few seconds to a few minutes are reasonable. Let the users be aware of the difference in expectations. However, in today's data warehouse and OLAP environments, response time beyond a few minutes is not acceptable. Strive to improve performance to keep the response time at this level. Ensure that performance is monitored regularly and the data warehouse is kept fine-tuned.

Monitoring and improving performance must happen at different levels. At the foundational level, make sure attention is paid by appropriate staff to performance of the operating system. At the next level lies the performance of the DBMS. Monitoring and performance improvement at this level rests on the data warehouse administrator. The higher levels of logical database design, application design, and query formatting also contribute to the overall performance.

Ensure Scalability This is a key objective. As we have seen, the usage of the data warehouse escalates over time, with a sharper increase during the initial period. We have discussed this supergrowth in some detail. During the supergrowth period, it is almost impossible to keep up with the steep rise in usage.

As you have already observed, the usage increases on two counts. The number of users increases rapidly and the complexity of the queries intensifies. As the number of users increases, the number of concurrent users of the data warehouse also increases proportionately. Adopt methods to address the escalation in the usage of the data warehouse on both counts.

Manage Storage Why is managing storage a major objective of physical design? Proper management of stored data will boost performance. You can improve performance by storing related tables in the same file. You can manage large tables more easily by storing parts of the tables at different places in storage. You can set the space management parameters in the DBMS to optimize the use of file blocks.

Provide Ease of Administration This objective covers the activities that make administration easy. For instance, ease of administration includes methods for proper arrangement of table rows in storage so that frequent reorganization is avoided. Another area for ease of administration is in the back up and recovery of database tables. Review the various data warehouse administration tasks. Make it easy for administration whenever it comes to working with storage or the DBMS.

Design for Flexibility In terms of physical design, flexibility implies keeping the design open. As changes to the data model take place, it must be easy to propagate the changes to the physical model. Your physical design must have built-in flexibility to satisfy future requirements.

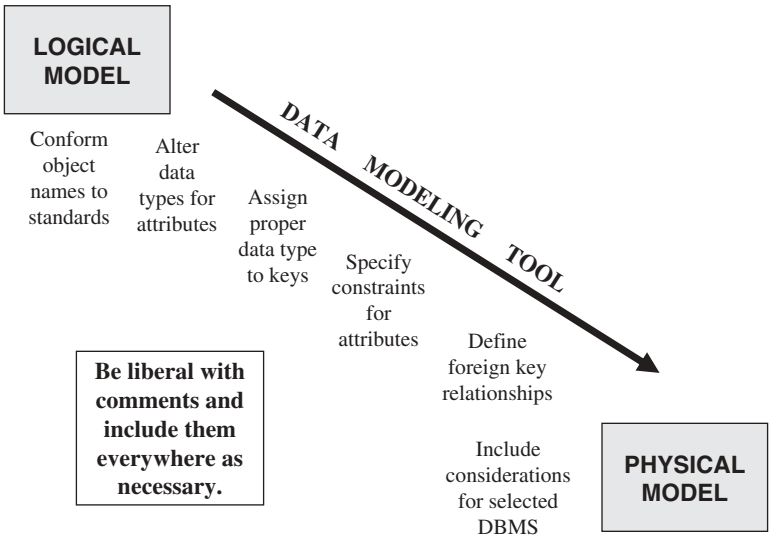


Figure 18-2 From logical model to physical model.

From Logical Model to Physical Model

In the logical model you have the tables, attributes, primary keys, and relationships. The physical model contains the structures and relationships represented in the database schema coded with the data definition language (DDL) of the DBMS. What are the activities that transform a logical model into a physical model? Figure 18-2 displays the activities marked alongside the arrow that follows the transformation process. At the end on the right side, notice the box indicated as the physical model.

This is the result of carrying out the activities mentioned alongside the arrow. Review this set of activities and adapt it for your data warehouse environment.

Physical Model Components

Having talked about the physical model in general terms and how to get to it through the physical design steps, let us now explore it in detail. The physical model represents the information content at a level closer to the hardware. That means you should have details such as file sizes, field lengths, data types, primary keys, and foreign keys all reflected in the model. Figure 18-3 indicates the major components of the physical model. Make note of the components, one by one. As you know, the components are described to the data dictionary (also known as the data catalog) of the DBMS through schemas and subschemas. You use the data definition language of the DBMS to write the schema definitions. Figure 18-4 gives you an example of schema definitions. Note the different types of schema statements. Notice the statements defining the database, the tables, and the columns within each table. Observe how data types and field sizes are defined. Those of you who have worked in database administration are quite familiar with the schema statements.

Let us tie it all together. Let us relate the components of the logical model to those of the physical model. Figure 18-5 presents such a combined view. Notice how these relate to the schema definition shown in Figure 18-4.

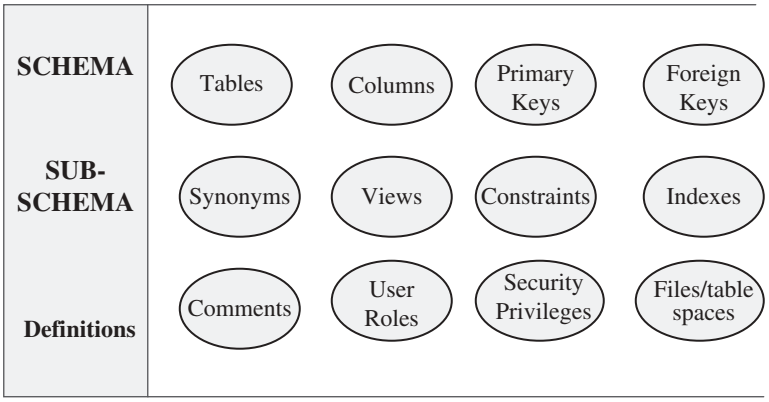


Figure 18-3 Components of the physical model of a data warehouse.

Significance of Standards

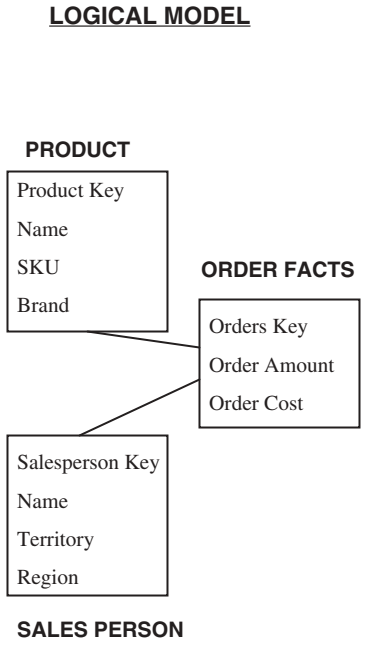
Standards in a data warehouse environment cover a wide range of objects, processes, and procedures. With regard to the physical model, the standards for naming the objects take on special significance. Standards provide a consistent means for communication. Effective communication must take place among the members of the project. In a data warehouse project, the presence of user representatives is more pronounced. As you know, users are more directly involved in accessing information from a data warehouse than they are in an OLTP environment. Clear communication with the users becomes more significant.

```
CREATE SCHEMA ORDER_ANALYSIS
  AUTHORIZATION SAMUEL_JOHNSON
  ……...
CREATE TABLE PRODUCT  (
  PRODUCT_KEY          CHARACTER (8)
                       PRIMARY KEY,
  PRODUCT_NAME         CHARACTER (25),
  PRODUCT_SKU          CHARACTER (20),
  PRODUCT_ BRAND       CHARACTER (25))

CREATE TABLE SALESPERSON  (
  SALPERS_KEY          CHARACTER (8)
                       PRIMARY KEY,
  SALPERS_NAME         CHARACTER (30),
  TERRITORY            CHARACTER (20),
  REGION               CHARACTER (20))

CREATE TABLE ORDER_FACT  (
  PRODUCT_REF          CHARACTER (8)
                       PRIMARY KEY,
  SALPERS_REF          CHARACTER (8)
                       PRIMARY KEY,
  ORDER_AMOUNT         NUMERIC (8,2),
  ORDER_COST           NUMERIC (8,2),
  FOREIGN KEY PRODUCT_REF
        REFERENCES PRODUCT,
  FOREIGN KEY SALPERS_REF
        REFERENCES SALESPERSON)
```

Figure 18-4 Sample schema definitions in SQL.



PHYSICAL MODEL

Table/Column Name	Data Type	Nulls allowed?	Comments
PRODUCT			Product dimension table including all of
			company's products
product_key	integer	N	Primary key
product_name		N	Name of product as used by marketing
product_sku		N	Stock Keeping Unit in source systems
product_brand		N	Name of brand as used by marketing
SALESPERSON			Sales Person dimension table includes all
			sales persons from all regions
salpers_key	integer	N	
salpers_name	char(30)	N	
territory	char(20)	N	
region	char(20)	N	
ORDER_FACT			Fact table containing metrics about all
			the orders received by the company
product_ref	integer	N	Partial primary key, also foreign key
			referencing product dimension table
salpers_ref	integer	N	Partial primary key, also foreign key
			referencing salesperson dimension table
order_amount	num (8,2)	N	Sales amount of order in dollars
order_cost	num (8,2)	N	Cost amount for the order in dollars

Figure 18-5 Logical model and physical model.

A few tips on standards follow.

Naming of Database Objects

Components of Object Names Have a clear method for composing names of the objects. The name itself must be able to convey the meaning and description of the object. For example, look at the name of a column: customer_loan_balance. This naming convention immediately identifies the column as containing values of balance amounts. What type of balance amounts? Loan balances. Is it a total loan balance amount? Whose loan balance amount? The first word denotes that it is the customer balance and not total balance. Object names made up of multiple words generally convey the meanings better. You can standardize the function of each word in the group of words indicating the name. In our example, the first word denotes the primary subject, the third word the general class of the object, and the second word qualifies the class. Many companies adopt this type of naming standard. You may take the standard already in use in your company and enhance it for clarity and conciseness.

Word Separators Standardize the separators that are also called the delineators. Dashes (-) or underscores (_) are commonly used. If your DBMS has specific conventions or requirements, follow those conventions.

Names in Logical and Physical Models Names for objects such as tables and attributes may include both the logical model versions and the physical model versions. You need naming standards for both versions. More than the user community, IT professionals

use the logical model names. Analysts and logical model designers communicate with each other through the logical model names. When the users need to refer to the tables and columns for data retrieval, they are communicating at the level of the physical model. Therefore, you need to adapt the standards for the physical model for the users. A better approach is to keep the logical and physical model versions of the name of a given object the same. If you need more qualifiers to further clarify the definition of the object, add them. Do not hesitate to have the definitions declared in business terminology.

Naming of Files and Tables in the Staging Area As you know, the staging area is a busy place in the data warehouse environment. A lot of data movement happens there. You create many intermediary files from the data extracted from the source systems. You transform and consolidate data in this area. You prepare the load files in the staging area. Because of the number of files in the staging area, it is easy to lose track of them. In order to avoid confusion, you have to be clear about which files serve what purposes. It is necessary to adopt effective standards for naming the data structures in the staging area. Consider the following suggestions.

Indicate the Process Identify the process to which the file relates. If the file is the output from a transformation step, let the name of the file denote that. If the file is part of the daily incremental update, let that be clear from the name of the file.

Express the Purpose Suppose you are setting up the scheduling of the weekly update to the product dimension table. You need to know the input load file for this purpose. If the name of the file indicates the purpose for which it was created, that will be a big help when you are setting up the update schedule. Develop standards for the staging area files to include the purpose of the file in the name.

Examples Given below are the names of a few files in the staging area. See if the following names are meaningful and the standards are adequate:

sale_units_daily_stage
customer_daily_update
product_full_refresh
order_entryinitial_extract
all_sources_sales_extract
customer_nameaddr_daily_update

Standards for Physical Files Your standards must include naming conventions for all types of files. These files are not restricted to data and index files for the data warehouse database. There are other files as well. Establish standards for the following:

- Files holding source codes and scripts
- Database files
- Application documents

PHYSICAL STORAGE

Consider the processing of a query. After the query is verified for syntax and checked against the data dictionary for authorization, the DBMS translates the query statements to determine what data is requested. From the data entries about the tables, rows, and columns desired, the DBMS maps the requests to the physical storage where the data access take place. The query gets filtered down to physical storage and this is where the input operations begin. The efficiency of the data retrieval is closely tied to where the data is stored in physical storage and how it is stored there.

What are the various physical data structures in the storage area? What is the storage medium and what are its characteristics? Do the features of the medium support any efficient storage or retrieval techniques? We will explore answers to questions such as these. From the answers you will derive methods for improving performance. First, let us understand the types of data structures in the data warehouse.

Storage Area Data Structures

Take an overall look at all the data related to the data warehouse. First, you have the data in the staging area. Though you may look for efficiency in storage and loading, arrangement of the data in the staging area does not contribute to the performance of the data warehouse from the point of view of the users. Looking further, the other sets of data relate to the data content of the warehouse. These are the data and index tables in the data warehouse. How you arrange and store these tables definitely has an impact on the performance. Next you have the multidimensional data in the OLAP system. In most cases, the supporting proprietary software dictates the storage and the retrieval of data in the OLAP system.

Figure 18-6 shows the physical data structures in the data warehouse. Observe the different levels of data. Notice the detail and summary data structures. Think further how the data structures are implemented in physical storage as files, blocks, and records.

Optimizing Storage

You have reviewed the physical storage structures. When you break each data structure down to the physical storage level, you find that the structure is stored as files in the physical storage medium. Take the example of the customer dimension and the salesperson dimension tables. You have basically two choices for storing the data of these two dimension tables. Store records from each table in one physical file. Or, if the records from these tables are retrieved together most of the time, then store records from both the tables in a single physical file. In either case, records are stored in a file. A collection of records in a file forms a block. In other words, a file comprises blocks and each block contains records.

In this subsection, let us examine a few techniques for optimizing storage. Remember any optimizing at the physical level is tied to the features and functions available in the DBMS. You have to relate the techniques discussed here with the workings of your DBMS. Study the following optimizing techniques.

Set the Correct Block Size As you understand, a set of records is stored in a block. What is special about the block? A data block in a file is the fundamental unit of input/output

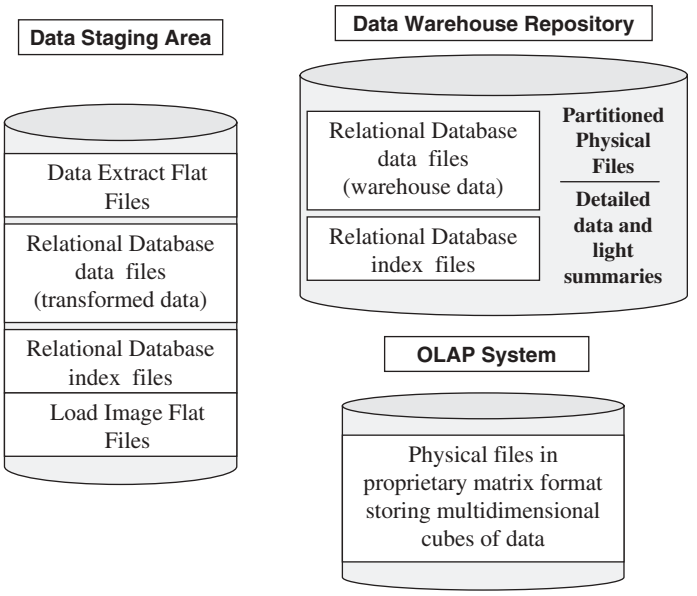


Figure 18-6 Data structures in the warehouse.

transfer from the database to memory where the data gets manipulated. Each block contains a block header that holds control information. The block header is not open to keeping data. Too many block headers means too much wasted space.

Assume that the block size for the customer file is 2 KB and that an average of 10 customer records can fit in a single block. Each DBMS has its own default block size—2 KB and 4 KB are common default block sizes. If the data records requested by a query reside in block number 10, then the operating system reads that entire block into memory to get the required data records.

What is the effect of increasing the block size in a file? More records or rows will fit into a single block. Because more records may be fetched in one read, larger block sizes decrease the number of reads. Another advantage relates to space utilization by the block headers. As a percentage of the space in a block, the block header occupies less space in a larger block. Therefore, overall, all the block headers put together occupy less space. But here is the downside of larger block sizes. Even when a smaller number of records are needed, the operating system reads too much extra information into memory, thereby impacting memory management.

However, because most data warehouse queries request large numbers of rows, memory management as indicated rarely poses a problem. There is another aspect of data warehouse tables that could cause some concern. Data warehouse tables are denormalized and therefore the records tend to be large. Sometimes a record may be too large to fit in a single block. Then the record has to be split across more than one block. The broken parts have to be connected with pointers or physical addresses. Such pointer chains affect performance to a large extent.

Consider all the factors and set the block size at the appropriate size. Generally, increased block size gives better performance but you have to find the proper size.

Set the Proper Block Usage Parameters Most of the leading DBMSs allow you to set block usage parameters at appropriate values and derive performance improvement. You will find that these usage parameters themselves and the methods for setting them are dependent on the database software. Generally, two parameters govern the usage of the blocks, and proper usage of the blocks enhances performance. Let us start with a generic example of these two parameters and then try to understand the implications.

Example:

Block Percent Free 20
Block Percent Used 40

Block Percent Free The DBMS leaves a percentage of each block empty so that the records in the block can expand into it. The records can use this reserved area in the block only when they expand on update. When a record is modified and expands, then the reserved area may be used. In the example, this parameter is set at 20. That means 20% of each block is reserved for expansion of records while being updated. In a data warehouse, there are hardly any updates. The initial load is all inserts of data records. The incremental loads are also mostly inserts. A few updates may take place while processing slowly changing dimension tables. Therefore, setting this parameter at a high value results in too much wasted space. The general rule is to set this parameter as low as possible.

Block Percent Used This parameter sets a watermark level below which the amount of space used in a block must fall before new records are accepted in that block. Take the example where this parameter is set at 40. As rows are deleted from a block, the freed space is not reused until at least 60% of the block is empty. Only when the amount of storage used falls below 40% is the freed space reused. What is the situation in a data warehouse? Mostly, addition of new records. There are hardly any deletes except when archiving out of the data warehouse. The general rule, therefore, is to set this parameter as high as possible.

Manage Data Migration When a record in a block is updated and there is not enough space in the same block for storing the expanded record, then most DBMSs move the entire updated record to another block and create a pointer to the migrated record. Such migration affects the performance, requiring multiple blocks to be read. This problem may be resolved by adjusting the block percent free parameter. However, migration is not a major problem in data warehouses because of the negligible number of updates.

Manage Block Utilization Performance degenerates when data blocks contain excessive amounts of free space. Whenever a query calls for a full table scan, performance suffers because of the need to read too many blocks. Manage block underutilization by adjusting the block percent free parameter downward and the block percent used parameter upward.

Resolve Dynamic Extension When the current extent on disk storage for a file is full, the DBMS finds a new extent and allows an insert of a new record. This task of finding a new extension on the fly is referred to as dynamic extension. However, dynamic extension comes with significant overhead. Reduce dynamic extension by allocation of large initial extents.

Employ File Striping Techniques You perform file striping when you split the data into multiple physical parts and store these individual parts on separate physical devices. File striping enables concurrent input/output operations and improves file access performance substantially.

Using RAID Technology

Redundant array of inexpensive disks (RAID) technology has become common to the extent that almost all of today's data warehouses make good use of this technology. These disks are found on large servers. The arrays enable the server to continue operation even while they are recovering from the failure of any single disk. The underlying technique that gives the primary benefit of RAID breaks the data into parts and writes the parts to multiple disks in a striping fashion. The technology can recover data when a disk fails and reconstruct the data. RAID is very fault tolerant. Here are the basic features of the technology:

- Disk mirroring—writing the same data to two disk drives connected to the same controller.
- Disk duplexing—similar to mirroring, except here each drive has its own distinct controller.
- Parity checking—addition of a parity bit to the data to ensure correct data transmission.
- Disk striping—data spread across multiple disks by sectors or bytes.

RAID is implemented at six different levels: RAID 0 through RAID 5.

Figure 18-7 gives you a brief description of RAID. Note the advantages and disadvantages. The lowest level configuration, RAID 0, will provide data striping. At the other end of the range, RAID 5 is a very valuable arrangement.

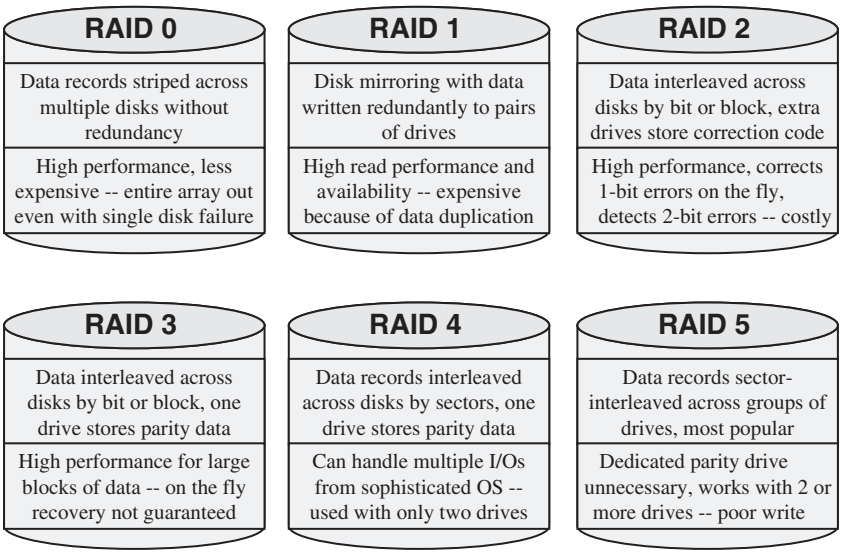


Figure 18-7 RAID technology.

Estimating Storage Sizes

No discussion of physical storage is complete without a reference to estimation of storage sizes. Every action in the physical model takes place in physical storage. You need to know how much storage space must be made available initially and on an ongoing basis as the data warehouse grows.

Here are a few tips on estimating storage sizes:

For each database table, determine

- Initial estimate of the number of rows
- Average length of the row
- Anticipated monthly increase in the number of rows
- Initial size of the table in megabytes (MB)
- Calculated table sizes in 6 months and in 12 months

For all tables, determine

- The total number of indexes
- Space needed for indexes, initially, in 6 months, and in 12 months

Estimate

- Temporary work space for sorting, merging
- Temporary files in the staging area
- Permanent files in the staging area

INDEXING THE DATA WAREHOUSE

In a query-centric system like the data warehouse environment, the need to process queries faster dominates. There is no surer way of turning your users away from the data warehouse than by unreasonably slow queries. For the user in an analysis session going through a rapid succession of complex queries, you have to match the pace of the query results with the speed of thought. Among the various methods to improve performance, indexing ranks very high.

What types of indexes must you build in your data warehouse? DBMS vendors offer a variety of choices. The choice is no longer confined to sequential index files. All vendors support B-Tree indexes for efficient data retrieval. Another option is the bitmapped index. As we will see later in this section, this indexing technique is very appropriate for the data warehouse environment. Some vendors are extending the power of indexing to specific requirements. These include indexes on partitioned tables and index-organized tables.

Indexing Overview

Let us consider the technique of indexing from the perspective of the data warehouse. The data tables are read-only. This feature implies that you almost never update the records or delete records. And records are not inserted into the tables after the loads. When you do adds, updates, or deletes, you incur additional overhead for manipulating the index files. But in a data warehouse this is not the case. So you can create a number of indexes for each table.

How many indexes can you create per table? Most of the indexing is done on the dimension tables. Generally, you will see a lot more indexes in a data warehouse than in an OLTP system. When a table grows in volume, the indexes also increase in size, requiring more storage. The general rule is that the maximum number of indexes varies inversely with the size of the table. Large numbers of indexes affect the loading process because indexes are created for new records at that time. You have to balance the various factors and decide on the number of indexes per table. Review the tables, one by one.

In the rest of this section, we will study specific indexing techniques in greater depth. Before doing so, please note the following general principles.

Indexes and Loading When you have a large number of indexes, the loading of data into the warehouse slows down considerably. This is because when each record is added to a data table, every corresponding index entry must be created. The problem is more acute during initial loads. You can address this problem by dropping the indexes before running the load jobs. By doing so, the load jobs will not create the index entries during the load process. After the loading process completes, you can run separate jobs to construct the index files. Construction of the index files takes substantial time, but not as much as creating index entries during the load process.

Indexing for Large Tables Large tables with millions of rows cannot support many indexes. When a table is too large, having more than just one index itself could cause difficulties. If you must have many indexes for the table, consider splitting the table before defining more indexes.

Index-Only Reads As you know, in the data retrieval process the index record is read first and then the corresponding data read takes place. The DBMS selects the best index from among the many indexes. Let us say the DBMS uses an index based on four columns in a table and many users in your environment request data from these four columns and one more column in the table. How does the data retrieval take place? The DBMS uses this index record to retrieve the corresponding data record. You need at least two input–output (I/O) operations. In this instance, the DBMS has to retrieve the data record just for one additional column. In such cases, consider adding that extra column to the index. The DBMS will read the index and find that all the information needed is contained in the index record itself, so it will not read the data record unnecessarily.

Selecting Columns for Indexing How do you select the columns in a table as most suitable for indexing? Which columns will produce the best performance if indexed? Examine the common queries and note the columns that are frequently used to constrain the queries. Such columns are candidates for indexing. If many queries are based on product line, then add product line to your list of potential columns for indexing.

A Staged Approach Many data warehouse administrators are puzzled about how to get started with indexing. How many indexes are needed for each table and which columns must be selected for indexing for the initial deployment of the data warehouse? They do a preliminary review of the tables, but have no experience with real-world queries yet. This is precisely the point. Experience cannot be a guideline. You need to wait for the users to exercise the data warehouse for some time. A staged approach to indexing seems to be prudent. Start with indexes on just the primary and foreign keys of each table.

Keep monitoring the performance carefully. Make a special note of any queries that run for an inordinately long time. Add indexes as more and more users come on board.

B-Tree Index

Most database management systems have the B-Tree index technique as the default indexing method. When you code statements using the data definition language of the database software to create an index, the system creates a B-Tree index. DBMSs also create B-Tree indexes automatically on primary key values. The B-Tree index technique is superior to other techniques because of its data retrieval speed, ease of maintenance, and simplicity. Figure 18-8 shows an example of a B-Tree index. Notice the tree structure with the root at the top. The index consists of a B-Tree (a balanced binary tree) structure based on the values of the indexed column. In the example, the indexed column is Name. This B-Tree is created using all the existing names that are the values of the indexed column. Observe the upper blocks that contain index data pointing to the next lower block. Think of a B-Tree index as containing hierarchical levels of blocks. The lowest-level blocks or the leaf blocks point to the rows in the data table. Note the data addresses in the leaf blocks.

If a column in a table has many unique values, then the selectivity of the column is said to be high. In a territory dimension table, the column for City contains many unique values. This column is therefore highly selective. B-Tree indexes are most suitable for highly selective columns. Because the values at the leaf nodes will be unique they will lead to distinct

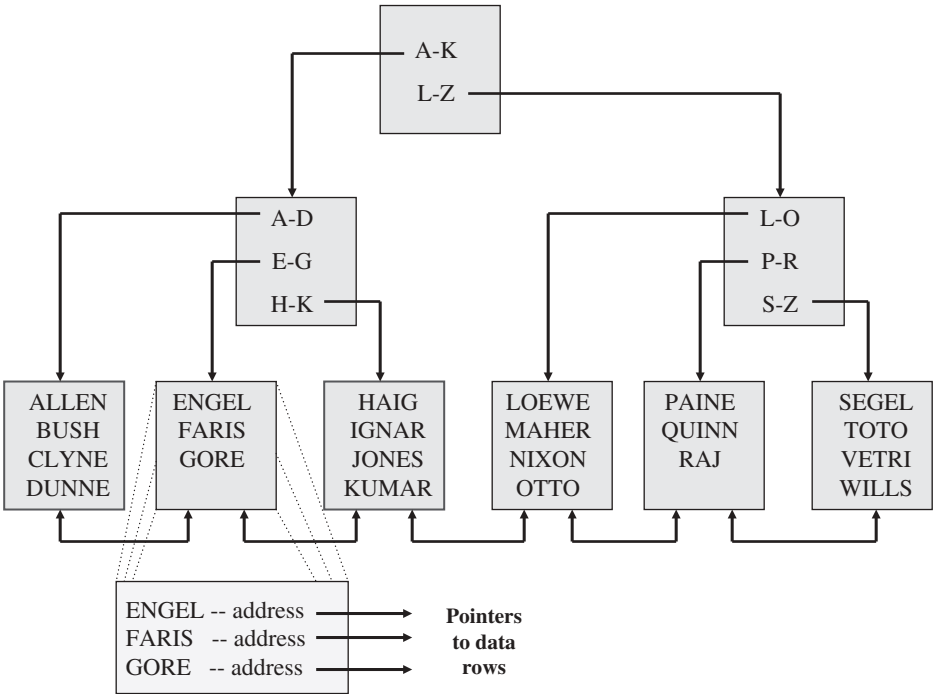


Figure 18-8 Example of a B-Tree index.

data rows and not to a chain of rows. What if a single column is not highly selective? How can you make use of B-Tree indexing in such cases? For example, the first name column in an employee table is not highly selective. Many common first names exist. But you can improve the selectivity by concatenating the first name with the last name. The combination is much more selective. Create a concatenated B-Tree index on both the columns together.

Indexes grow in direct proportion to the growth of the indexed data table. Wherever indexes contain concatenation of multiple columns, they tend to increase sharply in size. As the data warehouse deals with large volumes of data, the size of the index files can be cause for concern. What can we say about the selectivity of the data in the warehouse? Are most of the columns highly selective? Not really. If you inspect the columns in the dimension tables, you will notice a number of columns that contain low-selectivity data. B-Tree indexes do not work well with data whose selectivity is low. What is the alternative? That leads us to another type of indexing technique.

Extract of Sales Data

Address or Rowid	Date	Product	Color	Region	Sale($)
00001BFE.0012.0111	15-Nov-00	Dishwasher	White	East	300
00001BFE.0013.0114	15-Nov-00	Dryer	Almond	West	450
00001BFF.0012.0115	16-Nov-00	Dishwasher	Almond	West	350
00001BFF.0012.0138	16-Nov-00	Washer	Black	North	550
00001BFF.0012.0145	17-Nov-00	Washer	White	South	500
00001BFF.0012.0157	17-Nov-00	Dryer	White	East	400
00001BFF.0014.0165	17-Nov-00	Washer	Almond	South	575

Bitmapped Index for Product Column

Ordered bits: Washer, Dryer, Dishwasher

Address or Rowid	Bitmap
00001BFE.0012.0111	001
00001BFE.0013.0114	010
00001BFF.0012.0115	001
00001BFF.0012.0138	100
00001BFF.0012.0145	100
00001BFF.0012.0157	010
00001BFF.0014.0165	100

Bitmapped Index for Color Column

Ordered bits: White, Almond, Black

Address or Rowid	Bitmap
00001BFE.0012.0111	100
00001BFE.0013.0114	010
00001BFF.0012.0115	010
00001BFF.0012.0138	001
00001BFF.0012.0145	100
00001BFF.0012.0157	100
00001BFF.0014.0165	010

Bitmapped Index for Region Column

Ordered bits: East, West, North, South

Address or Rowid	Bitmap
00001BFE.0012.0111	1000
00001BFE.0013.0114	0100
00001BFF.0012.0115	0100
00001BFF.0012.0138	0010
00001BFF.0012.0145	0001
00001BFF.0012.0157	1000
00001BFF.0014.0165	0001

Figure 18-9 Example of a bitmapped index.

Bitmapped Index

Bitmapped indexes are ideally suitable for low-selectivity data. A bitmap is an ordered series of bits, one for each distinct value of the indexed column. Assume that the column for color has three distinct colors, namely, white, almond, and black. Construct a bitmap using these three distinct values. Each entry in the bitmap contains three bits. Let us say the first bit refers to white, the second to almond, and the third to black. If a product is white in color, the bitmap entry for that product consists of three bits, where the first bit is set to 1, the second bit is set to 0, and the third bit is set to 0. If a product is almond in color, the bitmap entry for that product consists of three bits, where the first bit is set to 0, the second bit is set to 1, and the third bit is set to 0. You get the picture. Now study the bitmapped index example shown in Figure 18-9. The figure presents an extract of the sales table and bitmapped indexes for the three different columns. Notice how each entry in an index contains the ordered bits to represent the distinct values in the column. An entry is created for each row in the base table. Each entry carries the address of the base table row.

How do the bitmapped indexes work to retrieve the requested rows? Consider a query against the sales table in the above example: Select the rows from the Sales table where product is "Washer" and color is "Almond" and division is "East" or "South."

Figure 18-10 illustrates how Boolean logic is applied to find the result set based on the bitmapped indexes shown in Figure 18-9.

As you may observe, bitmapped indexes support queries using low-selectivity columns. The strength of this technique rests on its effectiveness when using predicates on low-selectivity columns in queries. Bitmapped indexes take significantly less space than B-Tree indexes for low-selectivity columns. In a data warehouse, many data accesses are based on low-selectivity columns. Also, analysis using "what-if" scenarios require queries involving several predicates. You will find that bitmapped indexes are more suitable for a data warehouse environment than for an OLTP system.

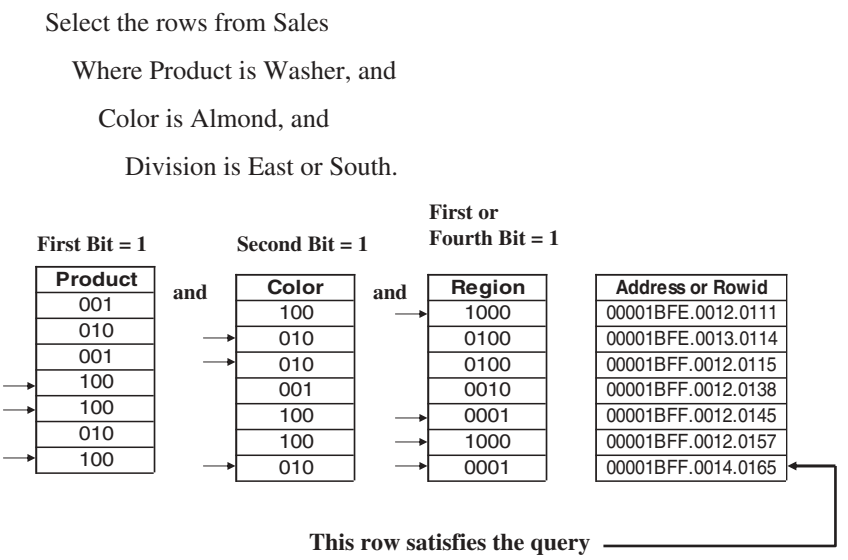


Figure 18-10 Bitmapped indexes: data retrieval.

On the other hand, if new values are introduced for the indexed columns, the bitmapped indexes have to be reconstructed. Another disadvantage relates to the necessity to access the data tables all the time after the bitmapped indexes are accessed. B-Tree indexes do not require table access if the requested information is already contained in the index.

Clustered Indexes

Some RDBMSs offer a new type of indexing technique. In the B-Tree, bitmapped, or any sequential indexing method, you have a data segment where the values of all columns are stored and an index segment where index entries are kept. The index segment repeats the column values for the indexed columns and also holds the addresses for the entries in the data segment. Clustered tables combine the data segment and the index segments; the two segments are one. Data is the index and index is the data.

Clustered tables improve performance considerably because in one read you get the index and the data segments. Using the traditional indexing techniques, you need one read to get the index segment and a second read to get the data segment. Queries run faster with clustered tables when you are looking for exact matches or searching for a range of values. If your RDBMS supports this type of indexing, make use of this technique wherever you can in your environment.

Indexing the Fact Table

What do you normally have inside a fact table? What is the nature of the columns? Revisit the STAR schema. The primary key of the fact table consists of the primary keys of all the connected dimensions. If you have four dimension tables of store, product, time, and promotion, then the full primary key of the fact table is the concatenation of the primary keys of the store, product, time, and promotion tables. What are the other columns? The other columns are metrics such as sales units, sales dollars, cost dollars, and so on. These are the types of columns to be considered for indexing the fact tables.

Please study the following tips and use them when planning to create indexes for the fact tables:

- If the DBMS does not create an index on the primary key, deliberately create a B-Tree index on the full primary key.
- Carefully design the order of individual key elements in the full concatenated key for indexing. In the high order of the concatenated key, place the keys of the dimension tables frequently referred to while querying.
- Review the individual components of the concatenated key. Create indexes on combinations based on query processing requirements.
- If the DBMS supports intelligent combinations of indexes for access, then you may create indexes on each individual component of the concatenated key.
- Do not overlook the possibilities of indexing the columns containing the metrics. For example, if many queries look for sales dollars within given ranges, then the column “sales dollars” is a candidate for indexing.
- Bitmapped indexing does not apply to fact tables. There are hardly any low-selectivity columns.

Indexing the Dimension Tables

Columns in the dimension tables are used in the predicates of queries. A query may run like this: How much are the sales of product A in the month of March for the northern division? Here the columns product, month, and division from three different dimension tables are candidates for indexing. Inspect the columns of each dimension table carefully and plan the indexes for these tables. You may be not be able to achieve performance improvement by indexing the columns in the fact tables but the columns in the dimension tables offer tremendous possibilities to improve performance through indexing.

Here are a few tips on indexing the dimension tables:

- Create a unique B-Tree index on the single-column primary key.
- Examine the columns that are commonly used to constrain the queries. These are candidates for bitmapped indexes.
- Look for columns that are frequently accessed together in large dimension tables. Determine how these columns may be arranged and used to create multicolumn indexes. Remember that the columns that are more frequently accessed or the columns that are at the higher hierarchical levels in the dimension table are placed at the high order of the multicolumn indexes.
- Individually index every column likely to be used frequently in join conditions.

PERFORMANCE ENHANCEMENT TECHNIQUES

Apart from the indexing techniques we have discussed in the previous section, a few other methods also improve performance in a data warehouse. For example, physically compacting the data when writing to storage enables more data to be loaded into a single block. That also means that more data may be retrieved in one read. Another method for improving performance is the merging of tables. Again, this method enables more data to be retrieved in one read. If you purge unwanted and unnecessary data from the warehouse in a regular manner, you can improve the overall performance.

In the remainder of this section, let us review a few other effective performance enhancement techniques. Many techniques are available through the DBMS, and most of these techniques are especially suitable for the data warehouse environment.

Data Partitioning

Typically, the data warehouse holds some very large database tables. The fact tables run into millions of rows. Dimension tables like the product and customer tables may also contain a huge number of rows. When you have tables of such vast sizes, you face certain specific problems. First, loading of large tables takes excessive time. Then, building indexes for large tables also runs into several hours. What about processing of queries against large tables? Queries also run longer when attempting to sort through large volumes of data to obtain the result sets. Backing up and recovery of huge tables takes an inordinately long time. Again, when you want to selectively purge and archive records from a large table, wading through all the rows takes a long time.

What if you are able to divide large tables into manageable chunks? Will you not see performance improvements? Performing maintenance operations on smaller pieces is easier

and faster. Partitioning is a crucial decision and must be planned up front. Doing this after the data warehouse is deployed and goes into production is time-consuming and difficult.

Partitioning means deliberate splitting of a table and its index data into manageable parts. The DBMS supports and provides the mechanism for partitioning. When you define the table, you can define the partitions as well. Each partition of a table is treated as a separate object. As the volume increases in one partition, you can split that partition further. The partitions are spread across multiple disks to gain optimum performance. Each partition in a table may have distinct physical attributes, but all partitions of the table have the same logical attributes.

What are the criteria for splitting a table into partitions? You can split a large table vertically or horizontally. In vertical partitioning, you separate out the partitions by grouping selected columns together. Each partitioned table contains the same number of rows as the original table. Usually, wide dimension tables are candidates for vertical partitioning. Horizontal partitioning is the opposite. Here you divide the table by grouping selected rows together. In a data warehouse, horizontal partitioning based on calendar dates works well. You can split a table into partitions of recent events and past history. This gives you the option to keep the recent events up and running while taking the historical component off-line for maintenance. Horizontal partitioning of the fact tables produces great benefits.

As you observe, partitioning is an effective technique for storage management and improving performance. Let us summarize the benefits:

- A query needs to access only the necessary partitions. Applications can be given the choice to have partition transparency or they may explicitly request an individual partition. Queries run faster when accessing smaller amounts of data.
- An entire partition may be taken off-line for maintenance. You can separately schedule maintenance of partitions. Partitions promote concurrent maintenance operations.
- Index building is faster.
- Loading data into the data warehouse is easy and manageable.
- Data corruption affects only a single partition. Backup and recovery on a single partition reduces downtime.
- The input–output load gets balanced by mapping different partitions to the various disk drives.

Data Clustering

In the data warehouse, many queries require sequential access of huge volumes of data. The technique of data clustering facilitates such sequential access. Clustering fosters sequential prefetch of related data.

You achieve data clustering by physically placing related tables close to each other in storage. When you declare a cluster of tables to the DBMS, the tables are placed in neighboring areas on the disk. How you exercise data clustering depends on the features of the DBMS. Review the features and take advantage of data clustering.

Parallel Processing

Consider a query that accesses large quantities of data, performs summations, and then makes a selection based on multiple constraints. It is immediately obvious that you will achieve major performance improvement if you can split the processing into components and execute the components in parallel. The simultaneous concurrent executions will

produce the result faster. Several DBMS vendors offer parallel processing features that are transparent to users. As a designer of the query, the user need not know how a specific query must be broken down for parallel processing. The DBMS will do that for the user.

Parallel processing techniques may be applied to data loading and data reorganization. Parallel processing techniques work in conjunction with data partitioning schemes. The parallel architecture of the server hardware also affects the way parallel processing options may be invoked. Some physical options are critical for effective parallel processing. You have to assess propositions like placing two partitions on the same storage device if you need to process them in parallel, Parallel processing and partitioning together provide great potential for improved performance. But the designer must decide how to use them effectively.

Summary Levels

As we have discussed several times, the data warehouse needs to contain both detailed and summary data. Select the levels of granularity for the purpose of optimizing the input–output operations. Let us say you keep the sales data at the levels of daily detail and monthly summaries. If the users frequently request weekly sales information, then consider keeping another summary at the weekly level. On the other hand, if you only keep weekly and monthly summaries and no daily details, any query for daily details cannot be satisfied from the data warehouse. Choose your summary and detail levels carefully based on user requirements.

Also, rolling summary structures are especially useful in a data warehouse. Suppose in your data warehouse you need to keep hourly data, daily data, weekly data, and monthly summaries. Create mechanisms to roll the data into the next higher levels automatically with the passage of time. Hourly data automatically gets summarized into the daily data, daily data into the weekly data, and so on.

Referential Integrity Checks

As you know, referential integrity constraints ensure the validity between two related tables. The referential integrity rules in the relational model govern the values of the foreign key in the child table and the primary key in the parent table. Every time a row is added or deleted, the DBMS verifies that the referential integrity is preserved. This verification ensures that parent rows are not deleted while child rows exist and that child rows are not added without parent rows. Referential integrity verification is critical in the OLTP systems, but it reduces performance.

Now consider the loading of data into the data warehouse. By the time the load images are created in the staging area, the data structures have already gone through the phases of extraction, cleansing, and transformation. The data ready to be loaded has already been verified for correctness as far as parent and child rows are concerned. Therefore, there is no further need for referential integrity verification while loading the data. Turning off referential integrity verification produces significant performance gains.

Initialization Parameters

DBMS installation signals the start of performance improvement. At the start of the installation of the database system, you can carefully plan how to set the initialization parameters. Many times you will realize that performance degradation is to a substantial extent the result of inappropriate parameters. The data warehouse administrator has a special responsibility to choose the right parameters.

For example, if you set the maximum number of concurrent users too low, the users will run into bottlenecks. Some users may have to wait to get into the database even though resources are available, simply because the parameter was set too low. On the other hand, setting this parameter too high results in unnecessary consumption of resources. Next, consider the checkpoint frequency. How often must the DBMS write checkpoint records? If the range between two consecutive checkpoints is too narrow, too much system resources will be used up. Setting the range too wide may affect recovery. These are just a couple of examples. Review all the initialization parameters and set each appropriately.

Data Arrays

What are data arrays? Suppose in a financial data mart you need to keep monthly balances of individual line accounts. In a normalized structure, the monthly balances for a year will be found in 12 separate table rows. Assume that in many queries users request the balances for all the months together. How can you improve the performance? You can create a data array or repeating group with 12 slots, each to contain the balance for one month.

Although creating arrays is a clear violation of normalization principles, this technique yields tremendous performance improvement. In the data warehouse, the time element is interwoven into all data. Frequently, users look for data in a time series. Another example is the request for monthly sales figures for 24 months for each salesperson. If you analyze the common queries, you will be surprised to see how many need data that can be readily stored in arrays.

CHAPTER SUMMARY

- Physical design takes the data warehouse implementation closer to the hardware. Physical design activities may be grouped into seven distinct steps.
- The importance of standards cannot be overemphasized. Adopt sound standards during the physical design process.
- Optimizing storage allocation ranks high in the physical design activities. Make use of RAID technology.
- Data warehouse performance is heavily dependent on proper indexing strategy. B-Tree indexes and bitmapped indexes are suitable.
- Other performance improvement schemes that are part of the physical design include the following: data partitioning, data clustering, parallel processing, creation of summaries, adjusting referential integrity checks, proper setting of DBMS initialization parameters, and use of data arrays.

REVIEW QUESTIONS

1. Name the physical design objectives. Which objective do you think ranks as the most important?
2. What are the components that make up the physical model? How are these related to components of the logical model?

3. Give two reasons why naming standards are important in a data warehouse environment.
4. List any three techniques for optimizing storage. Describe these briefly.
5. What is index-only read? How does it improve performance?
6. Give two reasons why B-Tree indexing is superior to other indexing methods.
7. What is meant by the selectivity for a column in a physical table? What type of indexing technique is suitable for low-selectivity data? Why?
8. What is data partitioning? Give two reasons why data partitioning is helpful in a data warehousing environment.
9. What is data clustering? Give an example.

EXERCISES

1. Match the columns:

1. fact table	**A.** set at high level
2. dynamic extension	**B.** data address in each entry
3. file striping	**C.** repeating data group
4. block percent used	**D.** combined data and index segments
5. B-Tree index	**E.** DBMS finds new extent
6. referential integrity check	**F.** set at low level
7. clustered index	**G.** candidate for partitioning
8. block percent free	**H.** data addresses in leaf nodes
9. bitmapped index	**I.** store on separate devices
10. data array	**J.** suspend for loading

2. Prepare an outline for a standards manual for your data warehouse. Consider all types of objects and their naming conventions. Indicate why standards are important. Produce a detailed table of contents.
3. Refer back to the STAR schema for orders analysis shown in Figure 10.7. Transform it into a physical model. Show all the components of the physical model. Relate the physical model to the logical model.
4. What are the two common block usage parameters? Using appropriate examples, describe how storage utilization may be improved in the data warehouse by setting these parameters properly. How are the settings different from OLTP systems? Why?
5. As the data warehouse administrator, performance enhancement is high on your list. Highlight the techniques you plan to adopt. For each technique, indicate tasks necessary to implement the technique.